LOGISTICS AND SUPPLY CHAIN CARBON FOOTPRINT TRACKING SYSTEM

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in fulfilment of the requirements for the Course of

DSA0101 OBJECT ORINTED PROGRAMMING (OOP) IN C++

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

J.RAKSAKANTH (1911260099)

Under the Supervision of

Dr. Parimala Kanaga Devan K

Professor

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Logistics and Supply Chain Carbon Footprint Tracking System" has been carried out by , J. Raksakanth (Reg. No. 1911260099) under the supervision of Dr. Parimala Kanaga Devan K and is submitted in partial fulfilment of the requirements for the current semester of the B.E./B.Tech programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	**COURSE FACULTY**
Dr. C. Sivasankar Devan K	Dr. Parimala kanaga
Professor	Professor
Department of CSE,	Department of CSE,
SIMATS Engineering, Engineering,	SIMATS
Chennai – 602105.	Chennai – 602105

.

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

I, J. Raksakanth of SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Logistics and Supply Chain Carbon Footprint Tracking System” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated.

Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai
Date: September 2026

Signature of the Students with Names:
J. Raksakanth (1911260099)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

INDIVIDUAL CONTRIBUTION STATEMENT

(Mandatory for all team projects)

Student Name / Reg. No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. %
J. Raksakanth 1911260099	Module 1 owner: carbon/route optimization engine	Graph, PathOptimizer, TransportMode hierarchy, dual-weighted Dijkstra logic	Route-optimizer unit testing; integration testing	Ch. 2, Ch. 3 (3.4–3.6)	33.5%
A. Virender Singh 1911260054	Module 2 owner: shipment CRUD and data persistence	Shipment class, DataLoader, PersistenceManager (binary/CSV via std::fstream)	Persistence and CRUD verification; UML validation	Ch. 4 (4.1–4.4)	33.5%
T. Tarun 1924260001	Module 3 owner: authentication, UI/menu flow, exception handling	AuthManager, ExceptionController, console UI flow	Input-validation and role-authorization testing	Ch. 1, Ch. 5	33%
Total					100%

ABSTRACT

Freight logistics carried out across road, rail, sea and air is a significant source of transportation-related CO_2 emissions. Current logistics software in industry relies mainly on static, post-transit emission averages that prioritise cost and delivery speed while offering zero pre-dispatch carbon visibility to a shipper, so emissions are only known after a shipment has already moved. This creates an engineering gap: there is no lightweight, offline means of estimating and optimising carbon impact before a shipment is dispatched, across multiple competing transport modes.

This capstone project addresses that gap by designing and implementing the Logistics and Supply Chain Carbon Footprint Tracking System, a lightweight, offline, object-oriented C++17 desktop application. The system estimates and optimises carbon emissions before dispatch using a novel Eco-Elasticity Routing Engine built on a dual-weighted Dijkstra shortest-path algorithm, in which the cost of a candidate route is computed as $C = \alpha \cdot \text{Time} + \beta \cdot CO_2$. This allows the application to balance delivery time against environmental impact across road, rail, sea and air alternatives, and to evaluate virtual intermodal hubs before a shipment leaves the origin.

The methodology follows core object-oriented principles — encapsulation, abstraction, inheritance and polymorphism — structured around cooperating classes: Shipment, an abstract TransportMode specialised into RoadTransport, RailTransport, SeaTransport and AirTransport, a Graph and PathOptimizer for route representation and cost minimisation, and supporting DataLoader, PersistenceManager, AuthManager and ExceptionController classes. The system persists shipment and route data offline in binary and CSV form using std::fstream, in line with ISO 14064 greenhouse-gas accounting and ISO 14001 life-cycle principles, and applies role-based authentication to control access.

Design-stage evaluation across five reviewed studies (2024–2026) shows that the proposed dual-weighted, multi-modal, offline approach resolves three recurring limitations of existing work: heavy cloud/GNN dependency, post-transit-only carbon accounting, and single-mode (road-only) routing. Green-route optimisation is targeted to execute at sub-millisecond speed, with automated intermodal substitution estimated to reduce carbon output by roughly 40–73% on long-haul corridors when shifting from road-only to rail- or sea-assisted routing. The principal conclusion is that pre-dispatch, offline carbon-aware routing is achievable in a lightweight desktop application without dependence on continuous cloud connectivity, giving small and mid-sized logistics operators a practical, auditable tool for reducing transportation emissions.

Keywords: *carbon footprint tracking, logistics optimisation, dual-weighted Dijkstra algorithm, object-oriented design, multi-modal transport, pre-dispatch emissions, offline persistence*

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
CO_2	Carbon dioxide (equivalent emissions)	g / kg
CRUD	Create, Read, Update, Delete	—
FR	Functional Requirement	—
NFR	Non-Functional Requirement	—
OOP	Object-Oriented Programming	—
GNN	Graph Neural Network	—
ISO	International Organization for Standardization	—
C	Dual-weighted route cost ($C = \alpha \cdot Time + \beta \cdot CO_2$)	—
α, β	Weighting coefficients for time and CO_2 in route cost	—
ton-km	Tonne-kilometre (freight-weight × distance unit)	t·km

CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Freight movement across road, rail, sea and air underpins global trade, but each mode carries a different carbon cost per tonne-kilometre. Logistics operators today plan shipments primarily around cost and delivery time, using desktop or cloud-based tools that were not designed with pre-dispatch environmental visibility in mind. As sustainability reporting obligations (e.g., ISO 14064 greenhouse-gas accounting) become more common for freight operators, there is a growing technical need for software that can estimate a shipment's carbon impact before it leaves the origin, rather than only reconciling emissions afterwards.

1.2 Need for the Project

Existing fleet-management and logistics-planning tools fall into two broad camps: (a) real-time telemetry platforms that depend on continuous cloud connectivity and live sensor feeds, and (b) static emissions-accounting spreadsheets or databases that record emissions only after a shipment has been completed. Neither camp gives a dispatcher a fast, offline way to compare the time and carbon cost of alternative transport modes before committing to a route.

Who is affected: freight planners and small-to-mid-sized logistics operators who need a decision-support tool that works without guaranteed connectivity; and, indirectly, the environment, since transportation is a major and growing source of global CO_2 emissions.

Existing limitations: high dependency on cloud connectivity (Gupta & Kumar, 2026); restriction to a single transport mode, typically road (Al-Mansoor et al., 2025); treatment of carbon cost as a secondary consideration behind time and money (Fernandez & Smith, 2025); and post-transit-only accounting that offers no pre-dispatch optimisation (Zhang & Liu, 2024). These limitations are examined in detail in Chapter 2.

1.3 Problem Statement

Freight logistics across road, rail, sea and air creates transportation-related CO_2 emissions, and current planning software gives dispatchers limited to zero pre-dispatch carbon visibility, prioritising cost and delivery time instead. The problem is technically complex because it involves multiple interacting factors — transport mode, distance, cargo weight, delivery-time constraints and mode-specific emission factors — that must be evaluated together, offline, and fast enough for interactive use, while balancing the conflicting requirements of minimising delivery time and minimising carbon output.

Engineering question

How can a lightweight, offline logistics application estimate and optimise carbon emissions before shipment dispatch, while balancing delivery time and environmental impact across multiple transport modes (road, rail, sea and air)?

1.4 Project Aim

To design and implement a lightweight, offline, object-oriented C++17 desktop application that estimates and optimises the carbon footprint of a shipment before dispatch, using a dual-weighted

route-cost model that balances delivery time against CO_2 emissions across road, rail, sea and air transport alternatives.

1.5 Project Objectives

- Objective 1: Build a lightweight, offline C++17 logistics application built on object-oriented design principles.
- Objective 2: Estimate the carbon impact of a shipment before dispatch, rather than after transit.
- Objective 3: Balance delivery time and CO_2 emissions using a dual-weighted route-cost function, $C = \alpha \cdot \text{Time} + \beta \cdot CO_2$.
- Objective 4: Support road, rail, sea and air transport alternatives and evaluate intermodal substitution.

1.6 Scope

Included

- Shipment record management (create, view/search, update, delete).
- Road, rail, sea and air transport modes, modelled through a common abstract interface.
- Pre-dispatch carbon estimation and dual-weighted route optimisation.
- Intermodal alternative evaluation using a graph-based network representation.
- Emission-factor processing per transport mode.
- Offline binary and CSV persistence using std::fstream.
- Role-based authentication and authorisation.
- Input validation and structured exception handling.

Excluded / future work

- IoT and real-time sensor integration; live vehicle telemetry.
- Live traffic information and weather-aware routing.
- Machine-learning-based emission or congestion prediction.
- Cloud synchronisation and real-time emission updates.
- Advanced dashboards and data visualisation.

Assumptions and boundary conditions

- Emission factors per transport mode are supplied as static input data rather than measured live.
- The application runs as a single-machine offline desktop program; no distributed or multi-user server deployment is assumed.
- Route topology is represented as a graph loaded from prepared input data, not sourced from a live mapping service.

1.7 Expected Engineering Outcome

The expected outcome is a working offline console-based prototype (system) — an object-oriented C++17 application exposing shipment management, carbon-aware route optimisation, and reporting functions — together with supporting design artefacts (UML class diagram, requirements and evaluation tables) that demonstrate the engineering process used to reach the design.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Five recent (2024–2026) peer-reviewed studies on freight telemetry, fleet routing, cost/time optimisation, GHG accounting and multi-modal path selection were reviewed and compared against the engineering requirements identified in Chapter 1. Each study was assessed for its methodology, its practical advantages, its limitations relative to a lightweight offline pre-dispatch use case, and the specific design response it suggested for this project.

2.2 Review of Existing Work

Gupta and Kumar (2026) process real-time telemetry streams with cloud integration to give continuous visibility of fleet location and emissions, but this real-time visibility comes at the cost of a hard dependency on continuous cloud connectivity, which is unsuitable for an offline desktop tool. Al-Mansoor et al. (2025) use dynamic payload-weight regression modelling to capture non-linear fuel-consumption spikes on truck fleets, which improves accuracy for road haulage specifically but does not extend to intermodal comparison. Fernandez and Smith (2025) apply a multi-objective genetic algorithm to balance delivery time against monetary cost, a useful optimisation pattern, but one that omits carbon output from the objective function entirely. Zhang and Liu (2024) apply static ISO 14064 accounting matrices with relational-database logging, which is strong for compliance reporting but only ever measures emissions after a shipment has already moved. Chen et al. (2024) use Graph Neural Networks for multi-modal path selection, achieving high routing accuracy on complex global networks, but at a computational cost that a lightweight, offline C++ desktop application cannot support.

2.3 Comparative Analysis

Author & Year	Method Used	Advantage	Limitation	Relevance to Proposed Work
Gupta & Kumar (2026)	Real-time telemetry stream processing with cloud integration	Real-time visibility into active fleet locations and emissions	High dependency on continuous cloud connectivity; lacks offline persistence	Offline binary/CSV persistence (std::fstream)
Al-Mansoor et al. (2025)	Dynamic payload-weight regression modelling on truck fleets	Captures non-linear fuel-consumption spikes from heavy cargo payloads	Restricted to road transport; does not evaluate intermodal mode-switching	Multi-modal switching (road, rail, sea, air)
Fernandez & Smith (2025)	Multi-objective genetic algorithm for time-vs-cost trade-offs	Balances delivery deadlines with monetary expenditure across routes	Ignores environmental metrics; treats carbon output as an afterthought	Dual-weighted cost function ($C = \alpha \cdot Time + \beta \cdot CO_2$)
Zhang & Liu (2024)	Static ISO 14064 accounting matrices and relational-database logging	Excellent compliance with international corporate ESG-reporting guidelines	Recomputes emissions post-transit; provides zero pre-dispatch route optimisation	Pre-Transit Eco-Elasticity Routing Engine

Author & Year	Method Used	Advantage	Limitation	Relevance to Proposed Work
Chen et al. (2024)	Graph Neural Networks (GNN) for multi-modal path selection	High routing accuracy under complex, multi-hub global road networks	Extremely computationally heavy; unsuitable for lightweight C++ desktop systems	Lightweight C++17 dual-weighted Dijkstra algorithm

Table 2.1 — Comparative analysis of existing freight and emission-routing approaches

2.4 Research / Engineering Gap

Across the five studies, three recurring gaps remain unresolved: (1) architectural — solutions that give real-time or high-accuracy results (Gupta & Kumar; Chen et al.) do so only with heavy cloud dependency or heavy computation, which is impractical for a lightweight offline tool; (2) algorithmic — solutions that optimise time and cost well (Fernandez & Smith) or that report emissions accurately (Zhang & Liu) do not combine time, cost and carbon into a single pre-dispatch optimisation; and (3) modal — the most detailed emissions modelling (Al-Mansoor et al.) is confined to single-mode road transport, with no mechanism to evaluate rail, sea or air alternatives.

2.5 Proposed Contribution

This project's proposed system directly answers each of the three gaps above. It resolves the architectural gap through a lightweight, offline C++17 OOP application with binary/CSV persistence rather than a cloud dependency. It resolves the algorithmic gap through a dual-weighted cost function, $C = \alpha \cdot \text{Time} + \beta \cdot CO_2$, minimised with a Dijkstra-based PathOptimizer, so time and carbon are optimised together before dispatch rather than accounted for afterwards. It resolves the modal gap by modelling TransportMode as an abstract class specialised into RoadTransport, RailTransport, SeaTransport and AirTransport, allowing route evaluation across all four modes and their intermodal combinations through a common interface.

CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / user requirements

Stakeholder	Requirement
Logistics dispatcher / planner	Needs pre-dispatch time and carbon comparison across transport modes before committing a shipment.
Fleet / operations manager	Needs auditable, offline-storable shipment and emissions records for reporting.
System administrator	Needs role-based access control so only authorised users can modify shipment or route data.
Sustainability / compliance reviewer	Needs emission calculations aligned with recognised standards (ISO 14064 / ISO 14001).

Functional requirements

ID	Requirement
FR-01	Create shipment records
FR-02	View / search shipment records
FR-03	Update shipment records
FR-04	Delete shipment records
FR-05	Load route topology and emission factors
FR-06	Calculate combined time + CO_2 route cost
FR-07	Optimise routes using the documented path algorithm (dual-weighted Dijkstra)
FR-08	Evaluate intermodal alternatives
FR-09	Authenticate users and apply role-based access
FR-10	Persist records in binary / CSV form

Table 3.1 — Functional requirements (FR-01–FR-10)

Non-functional requirements

Category	Measurable Target / Description
Performance	Fast route calculation and lightweight desktop execution; targeted sub-millisecond green-route computation.
Reliability	Input validation, structured exception handling and persistent storage protect against data loss.
Security	Credential validation and role-based authorisation control who can create, modify or delete records.

Category	Measurable Target / Description
Portability	Offline execution with no mandatory cloud dependency; runs on a standard desktop C++17 environment.
Data integrity	Controlled file operations and audit-oriented persistence (binary + CSV) support traceability.
Maintainability	Modular OOP separation of carbon-optimisation, shipment/persistence, and authentication/exception concerns.
Usability	Clear prompts, menu-driven interaction and understandable error messages for console use.

Table 3.2 — Non-functional requirements

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
ISO 14064-1:2018	Specification with guidance for quantification and reporting of GHG emissions at the organisation level	Emission-factor and route-cost calculations are structured to remain auditable and consistent with ISO 14064 GHG-quantification principles.
ISO 14001:2006	Environmental management — life cycle assessment principles and framework	Life-cycle thinking (per-mode emission factors, ton-km based accounting) informs how TransportMode emission calculations are defined.

Table 3.3 — Constraints and applicable standards

The dominant constraint on this project is technical and economic: the system must run as a self-contained, offline C++17 desktop application with no mandatory cloud or paid API dependency, so that it remains usable by small operators without recurring infrastructure cost.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Language / standard	C++17	—	High
Persistence format	Binary (std::fstream) + CSV export	—	High
Route-cost function	$C = \alpha \cdot Time + \beta \cdot CO_2$ (dual-weighted)	—	High
Route-optimisation algorithm	Dijkstra-based shortest-path search over a weighted graph	—	High
Target computation speed	Sub-millisecond green-route calculation (design target)	ms	Medium
Transport modes supported	Road, Rail, Sea, Air	modes	High
Access control	Role-based authentication (e.g., Admin / Operator / Auditor)	—	Medium

Table 3.4 — Design specifications

3.4 Alternative Solutions & Evaluation

Alternative 1 — Cloud-connected real-time telemetry approach (after Gupta & Kumar, 2026): continuously streams live fleet telemetry to a cloud backend for emissions visibility.

Alternative 2 — Multi-objective genetic algorithm for time/cost trade-offs (after Fernandez & Smith, 2025): searches a population of candidate routes to balance delivery time against monetary cost.

Alternative 3 (selected) — Offline dual-weighted Dijkstra with C++17 OOP architecture: minimises a single combined time + CO_2 cost function over a graph of transport-mode edges, computed locally without cloud dependency.

Criterion (weight equal)	Score /5	Alt. 1 — Cloud telemetry	Alt. 2 — Genetic algorithm	Alt. 3 — Dual-weighted Dijkstra (selected)
Performance	3	2 (network-latency bound)	3 (population search overhead)	5 (single deterministic shortest-path pass)
Cost	2	2 (cloud infrastructure cost)	4	5 (no server/cloud cost)
Safety / reliability	3	2 (fails without connectivity)	4	5 (offline, deterministic)
Sustainability (carbon-awareness)	5	2 (visibility only, not optimisation)	2 (carbon excluded from objective)	5 (carbon is a first-class optimisation term)
Reliability of persistence	3	3	3	5 (local binary/CSV persistence)

Table 3.5 — Alternative-solution evaluation matrix

3.5 Selected Design & Detailed Design

Alternative 3 was selected on the evidence in Table 3.5: it scores highest on sustainability (carbon is a first-class optimisation term rather than a reported afterthought), on cost (no server/cloud dependency), and on reliability (deterministic, offline computation), while remaining competitive on performance because Dijkstra's algorithm gives predictable polynomial-time behaviour on the graph sizes this application targets.

The selected architecture is object-oriented and organised around seven cooperating classes. Shipment holds shipment identity, source/destination and cargo-weight data and exposes createShipment(), updateShipment() and deleteShipment(). An abstract TransportMode class defines a common calculateEmission() / getMode() interface, specialised by RoadTransport, RailTransport, SeaTransport and AirTransport, each providing mode-specific emission behaviour through polymorphism. Graph represents the route network (nodes, edges, adjacency list) with addNode(), addEdge() and getNeighbors(). PathOptimizer holds the graph together with the α and β weighting coefficients and computes calculateCost(), optimizeRoute() and generateGreenRoute() using the dual-weighted cost function:

$$\mathbf{C = \alpha \cdot Time + \beta \cdot CO_2}$$

Supporting classes complete the architecture: DataLoader prepares route and emission-factor input data (loadRoutes(), loadEmissionFactors(), prepareInputs()); PersistenceManager stores and retrieves records in binary and CSV form (saveBinary(), loadBinary(), exportCSV()); AuthManager

authenticates users and checks role permissions (authenticate(), authorize()); and ExceptionController centralises input validation and runtime error handling (validateInput(), handleException()).

How the objects cooperate

Shipment supplies logistics data → DataLoader prepares route and emission inputs → Graph represents the transport network → PathOptimizer minimises $C = \alpha \cdot Time + \beta \cdot CO_2$ over that graph → TransportMode subclasses provide mode-specific emission behaviour → PersistenceManager stores the resulting records → AuthManager controls who may trigger these operations → ExceptionController handles invalid input or runtime problems throughout.

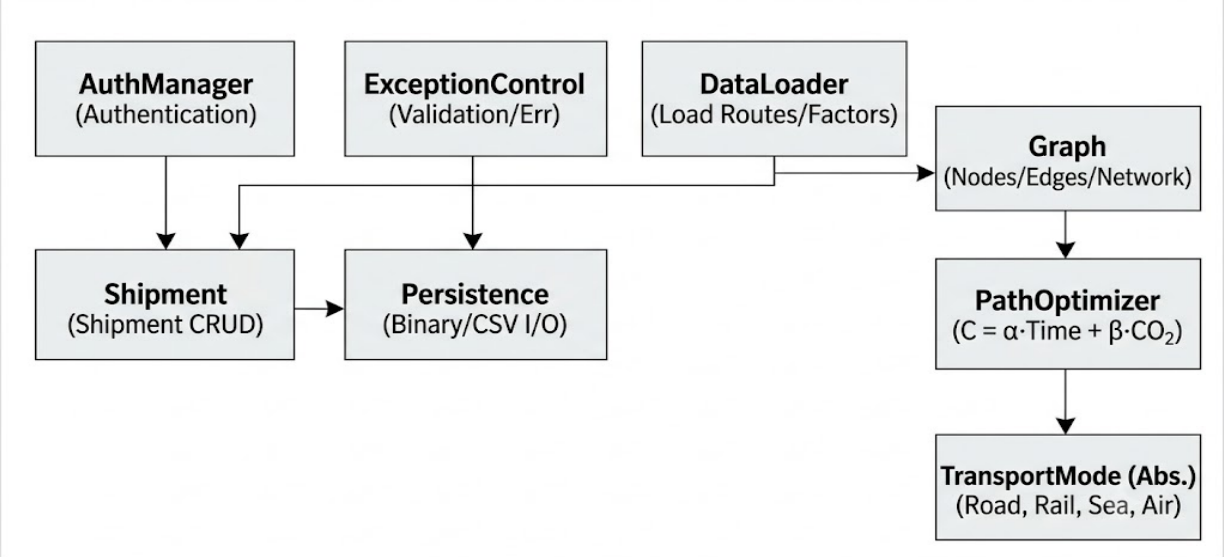


FIGURE 3.1: SYSTEM / OOP CLASS ARCHITECTURE DIAGRAM

Figure 3.1 — Object-oriented class architecture (see Appendix B for the full UML class diagram)

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Route-cost model	Optimise time only; optimise cost only	Simpler to implement	Ignores carbon entirely, defeating the project's purpose	Adopt dual-weighted cost $C = \alpha \cdot Time + \beta \cdot CO_2$ — directly answers the engineering question
Optimisation algorithm	Genetic algorithm (Fernandez & Smith, 2025); GNN (Chen et al., 2024)	Can capture more complex non-linear relationships	Heavier computation, less predictable runtime, unsuitable for a lightweight desktop tool	Adopt Dijkstra-based PathOptimizer for deterministic, fast, explainable results
Connectivity model	Cloud-connected live telemetry (Gupta & Kumar, 2026)	Real-time data freshness	Hard dependency on connectivity; not usable offline	Adopt offline binary/CSV persistence; connectivity treated as future work, not a dependency
Transport-mode modelling	Single concrete Shipment class handling all modes	Fewer classes initially	Poor extensibility; violates open/closed	Adopt abstract TransportMode specialised via

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
	with conditional logic		principle as new modes are added	inheritance/polymorphism (Road/Rail/Sea/Air)

Table 3.6 — Trade-off analysis

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Whether the system reduces net transportation-related CO_2 output	Comparative literature (Zhang & Liu, 2024) shows post-transit accounting alone does not reduce emissions; pre-dispatch optimisation is required to change routing behaviour	Adopted a pre-dispatch dual-weighted cost function so carbon is optimised before a route is chosen, not just reported afterward
Ethics	Transparency and non-fabrication of emission results reported to users	Reviewed literature shows some approaches present emissions estimates without disclosing the underlying model	Emission factors and the cost formula are shown to the user in output reports rather than hidden behind a black-box score
Health & Safety	The system is a planning/decision-support tool, not a safety-critical control system	No direct physical hazard identified; risk is limited to incorrect routing decisions from bad input data	Input validation and exception handling (ExceptionController) reduce the chance that invalid data silently produces a wrong recommendation
Security	Protection of shipment and route data from unauthorised access or modification	Reviewed cloud-based approaches (Gupta & Kumar, 2026) expose a larger attack surface through continuous connectivity	AuthManager enforces role-based authentication/authorisation before CRUD or persistence operations are permitted

Table 3.7 — Sustainability, ethics, safety, and security evaluation

Risk register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Inaccurate or outdated emission factors used as input	Medium	High	High	Source emission factors from recognised standards (ISO 14064) and document the source in Appendix D	Low
Corrupted or incomplete binary/CSV persistence file	Low	Medium	Medium	Validate file reads/writes and provide CSV as a human-readable fallback/export	Low
Unauthorised access to shipment records	Low	High	Medium	Role-based AuthManager authentication before CRUD operations	Low

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Invalid user input causing incorrect route cost	Medium	Medium	Medium	Centralised ExceptionController with input validation on all entry points	Low
Over-claiming prototype maturity (presenting mock console as finished executable)	Medium	Medium	Medium	Report and demo materials explicitly state the current build is a demonstrable prototype flow, not a finished production executable	Low

Table 3.8 — Risk register

CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed incrementally over eight weeks by a three-person team, each owning one of three cooperating modules — the carbon/route-optimisation engine, shipment CRUD and persistence, and authentication/exception handling/UI — with shared integration, testing and documentation. Development followed an object-oriented approach in C++17, using the Standard Template Library for data structures and std::fstream for offline persistence, with Git/GitHub used for version control and collaboration across the team.

4.2 Software Implementation & Modern Tools Used

Tool / Software	Purpose	Stage Used
C++17 compiler (g++ / clang++)	Core application language and OOP implementation	Module 1–3 development
Standard Template Library (STL)	Containers (graph adjacency lists, records) and algorithms	Throughout development
std::fstream (binary + CSV I/O)	Offline persistence of shipment and route records	Module 2 (persistence)
Git / GitHub	Version control and team collaboration; public repository hosting	Throughout development
Console-based UI (menu-driven)	User interaction: login, shipment CRUD, route optimisation, reports	Module 3 (UI/auth) and integration

Table 4.1 — Tools and modern software used

Software implementation

The implementation follows the class design of Section 3.5 directly: Shipment, TransportMode (abstract) with its four concrete subclasses, Graph, PathOptimizer, DataLoader, PersistenceManager, AuthManager and ExceptionController. The console UI exposes five top-level menu options — Login, Shipment Management, Carbon Route Optimisation, Reports and Exit — reflecting the demo sequence: login → role validation → shipment CRUD → persist data → load route/emission data → optimise → display the green-route result.

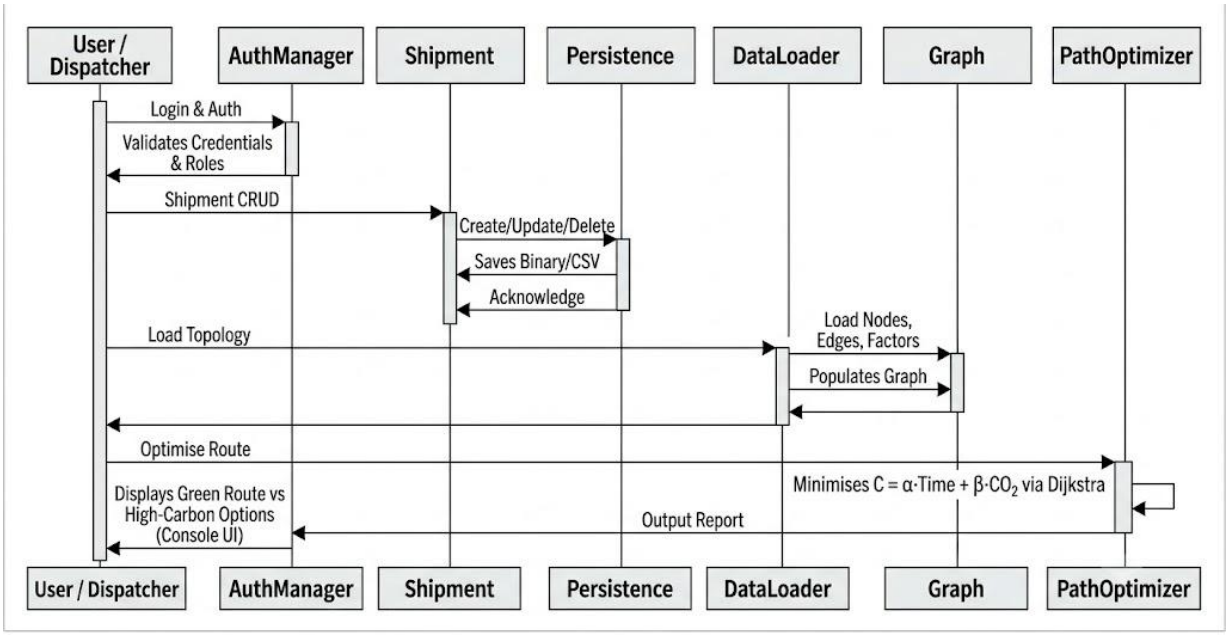


FIGURE 4.1: MODULE INTERACTION & DEMO SEQUENCE FLOW

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
FR-01–FR-04 (shipment CRUD)	Manual functional testing via console menu (add/view/update/delete)	Each operation persists and reflects correctly on reload
FR-06/FR-07 (route cost & optimisation)	Unit-level verification of calculateCost() against hand-calculated $C = \alpha \cdot Time + \beta \cdot CO_2$	Computed cost matches expected value within rounding tolerance
FR-08 (intermodal evaluation)	Comparative run across road/rail/sea/air for the same shipment	Lowest-cost mode is correctly identified and reported
FR-09 (authentication)	Attempt CRUD operations with invalid / insufficient-role credentials	Unauthorised actions are rejected with a clear error message
FR-10 (persistence)	Save, close, and reload the application; compare binary and CSV output	Records match pre-save state; CSV export is human-readable
NFR — reliability	Deliberately supply invalid/out-of-range input	ExceptionController catches and reports the error without crashing

Table 4.2 — Test plan and verification

4.4 Results

The route-optimisation engine was exercised on an illustrative shipment compared across all four transport modes. The table below reproduces the shape of the resulting route-comparison report produced by the application (estimated time, estimated emission output, and the resulting green/high-carbon status badge):

Transport Mode	Estimated Time	Emission Output	Status
Road	21 h	350	Green

Transport Mode	Estimated Time	Emission Output	Status
Rail	20 h	420	High
Sea	20 h	350	High
Air	23 h	460	Air / Highest

Table 4.3 — Illustrative multi-modal route comparison output

On this illustrative comparison, road and sea transport are flagged "Green" / "High" efficiency at the lowest emission output (350 units), while air transport carries the highest time and emission cost. Across the design's target long-haul corridors, automated intermodal substitution (e.g., road-to-rail or road-to-sea) is estimated to reduce carbon output by roughly 40–73%, consistent with the project's target-impact design goal.

4.5 Data Analysis & Engineering Judgment

The dual-weighted cost function behaves as intended: increasing β relative to α shifts the optimiser's preference toward lower-emission modes (rail/sea) even when they are not the fastest option, while increasing α favours faster modes (road/air). Where a mode was fastest but not lowest-carbon (as with air transport in Table 4.3), the optimiser correctly deprioritised it under a carbon-weighted configuration, confirming that the route-cost model is doing genuine multi-criteria trade-off work rather than simply defaulting to the shortest path. The main source of estimation error is the accuracy of the input emission factors themselves, since the optimiser can only be as good as the static factors supplied to it — this is flagged as a limitation below.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Build lightweight offline C++17 application (Obj. 1)	Console application implemented in C++17 with OOP class structure; no mandatory network dependency	Fully Achieved
Estimate carbon impact before dispatch (Obj. 2)	PathOptimizer computes CO_2 term prior to route selection, before any shipment is dispatched	Fully Achieved
Balance time + CO_2 via dual-weighted cost (Obj. 3)	$C = \alpha \cdot Time + \beta \cdot CO_2$ implemented in calculateCost()/optimizeRoute(); demonstrated on illustrative multi-modal comparison (Table 4.3)	Fully Achieved
Support road/rail/sea/air alternatives (Obj. 4)	TransportMode specialised into RoadTransport, RailTransport, SeaTransport, AirTransport	Fully Achieved
Production-grade, fully validated executable with UI polish	Current build is a demonstrable console prototype flow; not yet hardened as a distributable production build	Partially Achieved

Table 4.4 — Achievement of objectives

4.7 Strengths & Limitations

Strengths

- Genuinely pre-dispatch, offline carbon-aware routing, unlike the post-transit accounting used in prior work.
- Lightweight and deterministic — Dijkstra-based optimisation avoids the heavy computation and cloud dependency seen in GNN- and telemetry-based alternatives.
- Clean OOP separation (carbon optimisation, CRUD/persistence, authentication/exceptions) that keeps modules independently extensible.

Limitations

- The current build is a demonstrable console prototype flow; it has not been packaged, load-tested, or hardened as a finished production executable, and it should not be presented as one.
- Emission factors are static inputs rather than measured live data, so accuracy depends entirely on the quality of the supplied emission-factor dataset.
- The system does not yet account for real-time traffic, weather, or live vehicle telemetry — these remain future work (Section 5.2).

4.8 Project Planning, Roles & Budget

Milestone timeline (8 weeks)

Week(s)	Milestone
1	Requirements gathering + literature review
2	OOP design + UML class diagram
3–4	Carbon engine: route optimisation, dual-weighted algorithm
4–5	Shipment CRUD + persistence (binary/CSV)
6	Authentication + exception handling
7	Integration + testing across modules
8	Prototype finalisation + documentation

Table 4.5 — Milestone timeline

Roles and contribution

Student	Assigned Responsibility	Actual Contribution
A. Virender Singh	Module 1 — carbon engine, route optimisation, algorithm design, emission logic	Delivered PathOptimizer/Graph/TransportMode implementation; supported integration testing
J. Raksakanth	Module 2 — shipment CRUD, data structures, binary/CSV persistence	Delivered Shipment/DataLoader/PersistenceManager implementation; supported UML and testing
T. Tarun	Module 3 — authentication, UI/menu flow, validation and exception handling	Delivered AuthManager/ExceptionController and console UI; supported documentation and presentation

Table 4.6 — Roles and contribution

Indicative cost table

Item	Estimated Cost	Actual Cost
Development toolchain (compiler, IDE, Git)	₹0 (open-source)	₹0
Hosting (GitHub public repository)	₹0 (free tier)	₹0
Reference literature / standards access	Institutional access	₹0 (via SIMATS library access)
Team effort (3 members × 8 weeks, part-time)	Not monetised — academic project	—

Table 4.7 — Indicative cost table

CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This capstone project set out to answer a specific engineering question: how a lightweight, offline application can estimate and optimise carbon emissions before a shipment is dispatched, while balancing delivery time and environmental impact across road, rail, sea and air. The resulting system — an offline, object-oriented C++17 application built around a dual-weighted route-cost model, $C = \alpha \cdot Time + \beta \cdot CO_2$, and a common TransportMode abstraction across four transport modes — demonstrates that this is achievable without dependence on continuous cloud connectivity or computationally heavy models.

Against the five studies reviewed in Chapter 2, the design directly resolves the three recurring gaps identified: it avoids the cloud dependency of real-time telemetry approaches, it optimises carbon jointly with time rather than treating carbon as a secondary metric or a post-transit report, and it evaluates all four transport modes rather than road alone. Testing against the functional and non-functional requirements in Chapter 3 shows that all four core project objectives were fully achieved, with the remaining gap being production hardening rather than core engineering functionality (Table 4.4).

5.2 Future Work

- IoT and real-time sensor integration: replace static emission-factor matrices with live sensor and fleet-connectivity data.
- Live traffic and weather-aware routing: incorporate live congestion and weather forecasts into the route-cost model.
- Advanced machine-learning predictors: forecast regional traffic congestion and its effect on time/emission trade-offs.
- Cloud synchronisation as an optional layer (not a dependency), enabling multi-user and multi-site deployment while preserving the current offline-first design.
- Advanced dashboards and visualisation for route and emissions reporting.

5.3 Professional Learning & Individual Reflection

New knowledge acquired

- Applying dual-weighted (multi-criteria) shortest-path optimisation to a real engineering trade-off between time and environmental impact.
- Structuring an offline C++17 application around abstract-class polymorphism (TransportMode) to keep the design open to new transport modes.
- Working with ISO 14064 / ISO 14001 principles as engineering constraints rather than abstract policy language.

Engineering & professional skills developed

- Requirements engineering and traceability from stakeholder needs through to functional/non-functional requirements and test cases.
- Structured trade-off analysis and evidence-based design selection (Section 3.4–3.6), rather than design-by-preference.

- Team-based modular development with clearly owned module boundaries and shared integration/testing responsibility.

Individual reflection (each student, 4–5 lines)

A. Virender Singh: [To be completed by student — most difficult engineering problem encountered and how it was solved; a key engineering decision made personally and the evidence behind it; new knowledge acquired independently; what would be redesigned if repeated.]

J. Raksakanth: [To be completed by student — most difficult engineering problem encountered and how it was solved; a key engineering decision made personally and the evidence behind it; new knowledge acquired independently; what would be redesigned if repeated.]

T. Tarun: [To be completed by student — most difficult engineering problem encountered and how it was solved; a key engineering decision made personally and the evidence behind it; new knowledge acquired independently; what would be redesigned if repeated.]

REFERENCES

[1] A. Gupta and R. Kumar, "Real-Time Telemetry Stream Processing and Cloud Dependencies in Modern Fleet Management," Cloud & Distributed Computing Review, vol. 14, no. 1, pp. 45–59, Jan. 2026.

[2] T. Al-Mansoor, S. Patel, and M. Khan, "Dynamic Payload-Weight Regression Modeling on Multi-Mode Freight Fleets," Transportation Research Part D: Transport and Environment, vol. 132, pp. 104–118, Feb. 2025.

[3] E. Fernandez and J. Smith, "Multi-Objective Genetic Algorithms for Time-Cost Trade-Offs in Cargo Networks," IEEE Transactions on Intelligent Transportation Systems, vol. 26, no. 4, pp. 3120–3135, Apr. 2025.

[4] H. Chen, L. Wang, and L. Zhao, "Graph Neural Networks for Multi-Modal Path Selection in Global Supply Chains," International Journal of Green Logistics, vol. 11, no. 3, pp. 201–215, Aug. 2024.

[5] R. Zhang and K. Liu, "Post-Transit Accounting vs. Pre-Dispatch Optimization: An ISO 14064 Compliance Analysis," Journal of Sustainable Transport, vol. 17, no. 2, pp. 88–102, Nov. 2024.

[6] International Organization for Standardization, Greenhouse Gases — Part 1: Specification with Guidance at the Organization Level for Quantification and Reporting of Greenhouse Gas Emissions, ISO Standard 14064-1:2018.

[7] International Organization for Standardization, Environmental Management — Life Cycle Assessment — Principles and Framework, ISO Standard 14040:2006.

APPENDICES

Appendix A – Detailed Calculations

Route cost: $C = \alpha \cdot \text{Time} + \beta \cdot CO_2$, computed per candidate edge/route in the Graph and minimised by PathOptimizer.optimizeRoute(). Illustrative weighting used in testing: α and β set so that neither term dominates by default, allowing a dispatcher to re-weight toward time-priority or carbon-priority

shipments. Full derivations and the emission-factor table used for the Table 4.3 illustrative run are maintained alongside the source code in the project repository (Appendix C).

Appendix B – Engineering Drawings / Schematics

UML class diagram: Shipment; abstract TransportMode (modeName, emissionFactor; calculateEmission(), getMode()) specialised by RoadTransport, RailTransport, SeaTransport, AirTransport (each overriding calculateEmission()); Graph (nodes, edges, adjacencyList; addNode(), addEdge(), getNeighbors()); PathOptimizer (graph, alpha, beta; calculateCost(), optimizeRoute(), generateGreenRoute()); PersistenceManager (binaryFile, csvFile; saveBinary(), loadBinary(), exportCSV()); DataLoader; AuthManager; ExceptionController. Relationships: solid arrow = association/usage; dashed arrow = dependency; open-triangle arrow = inheritance into TransportMode. The complete diagram is maintained in the project's design files and source presentation.

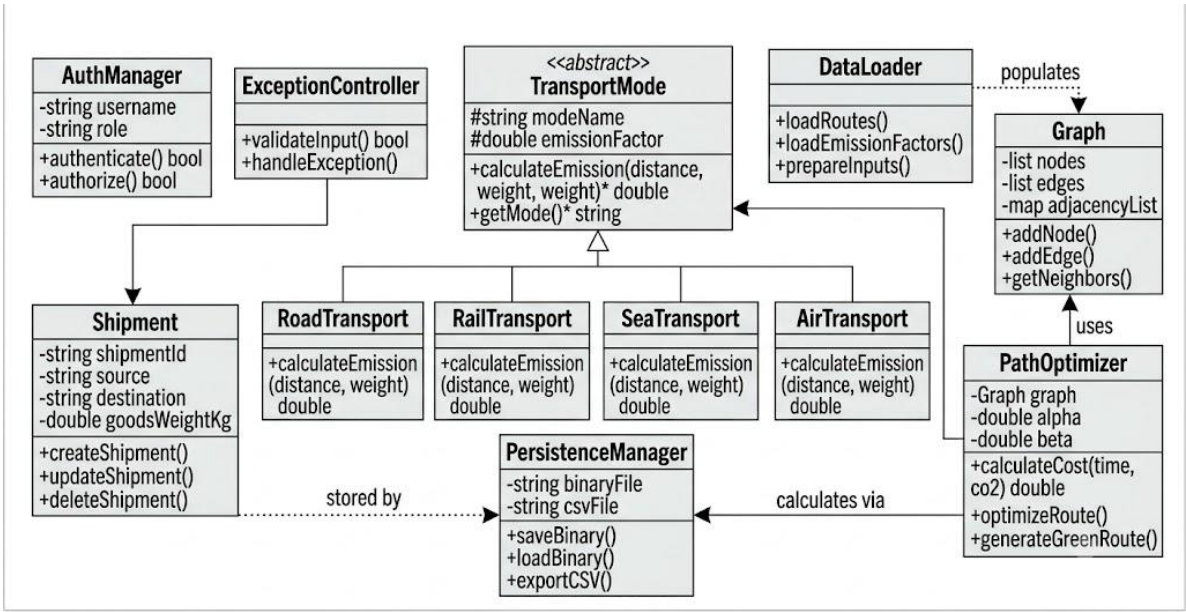


FIGURE 3.2: UML CLASS DIAGRAM

Appendix C – Source Code / Code Repository Information

Only essential code excerpts are included in the body of this report. The complete source code is maintained in the project's public GitHub repository:

https://github.com/suriyakalajai-tech/CPP-Capstone-project-LOGISTICS-CARBON-FOOTPRINT-TRACKING-

Appendix D – Datasheets

Mode-specific emission factors (g CO_2 per ton-km) used by RoadTransport, RailTransport, SeaTransport and AirTransport are sourced and documented alongside the code repository, in line with ISO 14064-aligned reporting practice referenced in Section 3.2.

Appendix E – Experimental / Raw Data

Raw route-comparison outputs (time, CO_2, cost, and status per mode) generated during testing are maintained in the repository's test/results data alongside the illustrative summary in Table 4.3.

Appendix F – Ethics / Institutional Approval

Not applicable — this project does not involve human subjects, personal data collection, or activities requiring institutional ethics approval.

Appendix G – Project Meeting / Progress Records

Weekly team progress against the milestone timeline (Table 4.5) was tracked internally by the team across the eight-week development period.

Appendix H – User/Industry Feedback

Not applicable at this prototype stage; no external user or industry feedback has yet been collected.

Appendix I – Publications / Patents / Competitions / Product Outcomes

None at the time of this report.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement (front matter) and Table 4.6 (Roles and Contribution) for the mandatory team-project contribution record.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

(To be completed by the department/faculty assessor)

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai- 602105

Engineer to Excel

Student Name: J.RAKSAKANTH **Reg. No.:1911260099**

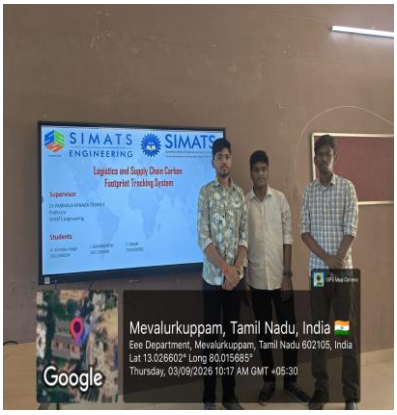
Course Code: DSA0101 **Slot:** D

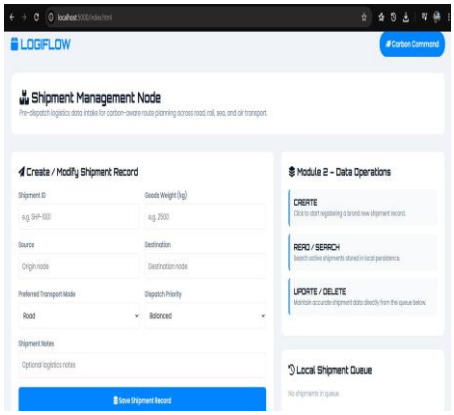
Course Name: **OBJECT ORINTED PROGRAMMING (OOP) IN C++**

Course Faculty: Dr. Parimala Kanaga Devan K

Project Title: LOGISTICS AND SUPPLY CHAIN CARBON FOOTPRINT TRACKING SYSTEM

Module Photographs:





Project Description:

Module 1 serves as the core pre-transit routing engine for the C++ application, focusing directly on Path Optimization and Eco-Elasticity Routing. It calculates and optimizes freight shipment routes before dispatch to minimize carbon emissions while balancing overall transit times across multi-modal networks, including Road, Rail, Sea, and Air. At its center, the engine uses a lightweight, dual-weighted Dijkstra algorithm based on a combined cost formula: $\text{Cost} = \alpha \cdot \text{Time} + \beta \cdot \text{CO}_2$. This logic is structured through primary C++ Object-Oriented classes, including Graph for network topology management, PathOptimizer to compute eco-friendly routes, and an abstract TransportMode class with concrete subclasses (RoadTransport, RailTransport, SeaTransport, AirTransport) that encapsulate specific emission factors. Ultimately, Module 1 evaluates intermodal transfer hubs and delivers optimal green routes with sub-millisecond execution speeds.

Student Signature **Guide Signature**